

bSecure Frontend Implementation

Overview

Three frontend applications for the bSecure security guard booking platform:

App	Location	Stack	Purpose
User App	bsecure/userapp/	Expo (React Native) + NativeWind + React Query + Zustand	Customer-facing mobile app
Guard App	bsecure/guardapp/	Expo (React Native) + NativeWind + React Query + Zustand	Security guard mobile app
Admin Panel	bsecure/adminpanel/	React + Vite + Tailwind + shadcn/ui + React Query	Web-based admin dashboard

Design System

Theme: Dark Minimalist Gen-Z

- **Background:** #1E1E2E (secondary/dark navy)
- **Surface/Cards:** #2A2A3E
- **Primary:** #6C63FF (purple — buttons, active states)
- **Accent:** #00D4AA (teal — success, online indicators)
- **Danger:** #FF4757 (red — SOS, errors, cancel)
- **Warning:** #FFA502 (amber)
- **Earnings:** #4CD137 (green — guard earnings)
- **Typography:** Inter font family, clean sans-serif
- **Corners:** rounded-2xl on cards, rounded-full on buttons/badges

- **Spacing:** Generous padding, minimal visual clutter
-

User App (bsecure/userapp/)

Setup

```
cd bsecure/userapp
npm install
npx expo start
```

Architecture

```
userapp/
├─ app/                                # Expo Router (file-based)
├─ routing)
│   └─ _layout.tsx                    # Root layout: providers, auth
├─ gate
│   └─ (auth)/                        # Auth stack
│       ├── login.tsx                 # Phone input (+91)
│       └─ otp-verify.tsx             # 6-digit OTP verification
│   └─ (tabs)/                        # Main tab navigator
│       └─ home.tsx                   # Map + bottom sheet + nearby
├─ guards
│   ├── bookings.tsx                  # Active/History booking list
│   ├── wallet.tsx                    # Balance, top-up, transactions
│   ├── profile.tsx                   # User profile, settings, logout
│   ├── booking/
│   │   ├── create.tsx                # Multi-step booking wizard
│   │   ├── tracking.tsx              # Live map + WebSocket tracking
│   │   └─ [id].tsx                   # Booking detail view
│   ├── guards/[id].tsx               # Guard profile modal
│   └─ sos.tsx                        # Hold-to-trigger SOS screen
├─ src/
│   └─ api/
│       └─ client.ts                  # Axios + JWT interceptors +
├─ refresh queue
│   └─ services/                      # auth, guards, bookings,
├─ wallet, user, sos
│   └─ stores/                        # Zustand: auth, booking,
├─ location
│   └─ hooks/                         # useNearbyGuards,
├─ useBookingWebSocket, useLocation, useWallet
├─ components/                       # GuardCard, BookingCard,
├─ OTPInput, StatusBadge, SOSButton, etc.
├─ types/index.ts                     # All TypeScript interfaces
├─ app.json                           # Expo config with plugins
├─ tailwind.config.js                 # NativeWind theme
└─ package.json
```

Key Features

- **Phone + OTP auth** with SecureStore token persistence
- **Google Maps** with dark styling and guard markers
- **Bottom Sheet** (Gorhom) for guard discovery
- **Real-time tracking** via WebSocket with reconnect logic
- **Wallet** with Razorpay integration for top-ups
- **SOS** hold-to-trigger with haptic feedback
- **Pull-to-refresh** on all list screens

State Management

- `useAuthStore` — JWT tokens in SecureStore, hydrate on app launch
- `useBookingStore` — active booking state
- `useLocationStore` — current GPS coordinates

API Integration

All services use the shared Axios client with automatic JWT injection and 401 token refresh queue.

Guard App (bsecure/guardapp/)

Setup

```
cd bsecure/guardapp
npm install
npx expo start
```

Architecture

```
guardapp/
├── app/
│   └── _layout.tsx           # Root: providers, auth gate,
onboarding gate
├── (auth)/
│   ├── welcome.tsx         # Welcome/branding screen
│   ├── login.tsx           # Phone input
│   └── otp-verify.tsx       # OTP verification
├── onboarding/
│   ├── personal-info.tsx    # Name, skills, experience form
│   └── documents.tsx        # Upload ID, police
verification, photo
├── complete.tsx            # Verification status timeline
└── (tabs)/
```

```

|   |   | dashboard.tsx      # Online toggle + radar + stats
|   |   | earnings.tsx      # Summary, chart, payout history
|   |   | history.tsx       # Filter pills + booking history
|   |   | profile.tsx       # Avatar, badge, stats, settings
|   |   | booking/
|   |   | | request.tsx      # Incoming request (30s
countdown)
|   |   | | active.tsx       # Full-screen map + step
progression
|   |   | | [id].tsx         # Booking detail + earnings
|   |   | src/
|   |   | | api/services/    # auth, profile, bookings,
earnings, documents, location, onboarding
|   |   | | stores/         # guard, activeBooking, earnings
|   |   | | hooks/          # useGuardStatus,
useIncomingRequest, useActiveBooking, etc.
|   |   | | services/websocket.ts # GuardWebSocketService
(singleton, reconnect)
|   |   | | lib/backgroundLocation.ts # TaskManager background GPS
tracking
|   |   | | components/      # OnlineToggle, RadarAnimation,
BookingStepBar, etc.
|   |   | | ...

```

Key Features

- **Background location tracking** via expo-task-manager (10s interval, sends via raw fetch)
- **Online/Offline toggle** with large animated pill
- **Incoming booking requests** via WebSocket with 30s auto-decline timer
- **Step progression** for active bookings (en_route → arrived → started → completed)
- **Radar animation** when online and waiting for requests
- **Earnings dashboard** with charts (victory-native)
- **Document upload** via S3 pre-signed URLs with progress tracking
- **Onboarding flow** with verification status polling

WebSocket Protocol

- **Receives:** booking.new_request, booking.request_cancelled, booking.status_changed, sos.alert
- **Sends:** location.update, booking.accepted, booking.declined, booking.arrived, booking.started, booking.completed

Background Location

Guard goes online → foreground location starts
 Booking accepted → background location tracking starts
 (TaskManager)

Booking completed → background tracking stops
Guard goes offline → all location tracking stops

Admin Panel (bsecure/adminpanel/)

Setup

```
cd bsecure/adminpanel
npm install
npm run dev
```

Architecture

```
adminpanel/
├── src/
│   ├── pages/
│   │   ├── Login.tsx          # Email/password auth
│   │   └── Dashboard.tsx      # KPI cards + charts + recent
│   ├── SOS
│   │   ├── guards/
│   │   │   ├── GuardsList.tsx # Filterable table
│   │   │   └── GuardDetail.tsx # Profile + docs + actions
│   │   ├── users/
│   │   │   ├── UsersList.tsx  # Searchable table
│   │   │   └── UserDetail.tsx  # Profile + history
│   │   ├── bookings/
│   │   │   ├── BookingsList.tsx # Status/date filtered table
│   │   │   └── BookingDetail.tsx # Timeline + amount breakdown
│   │   └── Payments.tsx        # Revenue KPIs + transactions +
│   ├── payouts
│   │   └── SOS.tsx             # Real-time SOS cards + alarm +
│   ├── history
│   │   ├── Analytics.tsx      # Line/bar/pie charts + heatmap
│   │   ├── Verifications.tsx  # Document review queue
│   │   └── Settings.tsx       # Platform configuration
│   ├── components/
│   │   └── ui/                 # shadcn/ui components (13
│   ├── components)
│   │   └── layout/             # Sidebar, Topbar,
│   ├── DashboardLayout
│   │   ├── KPICard.tsx
│   │   └── DataTable.tsx      # TanStack Table with
│   ├── pagination/sorting
│   │   ├── SOSEventCard.tsx
│   │   ├── BookingTimeline.tsx
│   │   └── ...
│   ├── api/services/          # 10 service modules
│   └── hooks/                  # useAdminWebSocket + React
└── Query hooks
```

├── stores/auth.ts	# Zustand auth store
├── types/index.ts	
├── tailwind.config.ts	# Dark theme with CSS variables
├── vite.config.ts	# Vite + path aliases
└── index.html	

Key Features

- **Real-time dashboard** with WebSocket-driven KPIs and SOS alerts
- **Guard management** — approve, suspend, change tier, view documents
- **Booking management** — force cancel, refund, view tracking history
- **SOS dashboard** — live cards with alarm sound + browser notifications
- **Analytics** — Recharts (line, bar, pie) + peak hours heatmap
- **Document verification** — approve/reject queue with reason dialog
- **Payment management** — revenue summary, transaction list, bulk payout approval
- **Platform settings** — fee percentages, hourly rates, payout thresholds
- **CSV export** on all data tables

WebSocket Events

- `sos.new / sos.resolved` — SOS alerts with alarm sound
 - `booking.new / booking.completed / booking.cancelled` — live booking metrics
 - `guard.online / guard.offline` — online count updates
 - `payment.new_transaction` — revenue updates
-

Shared Patterns

API Error Handling

All apps use a typed `ApiError` class that extracts Django's error envelope:

```
{
  error: {
    code: string;
    message: string;
    details?: Record<string, string[]>;
  }
}
```

Authentication Flow

1. Phone number input → Request OTP
2. 6-digit OTP verification → Receive JWT (access + refresh)

3. Tokens stored in SecureStore (mobile) / localStorage (web)
4. Axios interceptor auto-refreshes on 401

WebSocket Pattern

- Connect on auth success / relevant screen mount
 - Exponential backoff reconnect (max 30s delay, max 10 attempts)
 - Typed message envelopes: { type: string, payload: T }
 - Connection status indicator visible to user
-

Environment Variables

User App (.env)

```
EXPO_PUBLIC_API_URL=http://localhost:8000/api  
EXPO_PUBLIC_WS_URL=ws://localhost:8000/ws  
EXPO_PUBLIC_GOOGLE_MAPS_KEY=your_key  
EXPO_PUBLIC_RAZORPAY_KEY_ID=your_key
```

Guard App (.env)

```
EXPO_PUBLIC_API_URL=http://localhost:8000/api  
EXPO_PUBLIC_WS_URL=ws://localhost:8000/ws  
EXPO_PUBLIC_GOOGLE_MAPS_KEY=your_key
```

Admin Panel (.env)

```
VITE_API_BASE_URL=http://localhost:8000/api  
VITE_WS_URL=ws://localhost:8000/ws  
VITE_GOOGLE_MAPS_KEY=your_key
```